Save a Life

Informe del estado del proyecto · Save a Life

Campo	Valor
Proyecto	Save a Life: coordinación de hospitales y donantes de sangre (prototipo académico)
Fecha de corte	28 de octubre de 2026 (rama `feat/frontend-backend-integration`); la versión anterior del informe cortó el 22 de octubre de 2026 (rama `main`, commit `ea4ed5b`)
Equipo	Diego Roman (GitHub `DiegoRomanP`), Jhan Fernando Leiva Ignacio (`jignacio17`), `Ator512`
Repositorio	https://github.com/DiegoRomanP/Safe-a-Life
Gestión	Plane (vista pública, solo lectura): http://159.195.245.124:75/spaces/issues/83d844c48b9b40e1b51df54042cc00f3
Diagramas de clases y UML	https://github.com/DiegoRomanP/Safe-a-Life/tree/main/evidence_pc2
Diagramas de arquitectura y flujos	Eraser: https://app.eraser.io/workspace/iclH0MjPckR9i4f2cPaO
Documentación oficial	Sitio VitePress en `docs/` (`npm run docs:dev`, puerto 5170); ver la sección 9

1. Resumen

Save a Life conecta hospitales autorizados de Lima con posibles donantes de sangre compatibles y cercanos. El hospital publica una necesidad, el sistema filtra candidatos por grupo declarado compatible, disponibilidad y distancia, les avisa por correo y registra su **intención** de asistir. El hospital siempre verifica identidad, grupo y aptitud. La demo usa solo datos sintéticos (3 hospitales y hasta 500 donantes).

Estado general:

- El **backend** tiene implementadas 6 de sus 9 capacidades.
- El **frontend** tiene todas sus pantallas web, con datos de prueba.
- La **documentación** está organizada por libros y se publica como sitio con VitePress.
- **Falta** el módulo de invitaciones (F6), el de información legal (F8) y conectar y publicar el sistema (F9).

2. Avance por área

Para el avance se usó un criterio simple y explícito: cada capacidad o bloque vale lo mismo, y un bloque cuenta como hecho solo si tiene pruebas automáticas y está en `main`.

2.1 Backend (capacidades F1 a F9)

Capacidad	Qué cubre	Estado
F1 Plataforma	Salud del sistema, configuración, seguridad HTTP	✅ Completa
F2 Identidad e instituciones	Sesión, MFA, step-up, hospitales, sedes, administradores, auditoría	✅ Completa
F3 Necesidades	Crear, listar, ver y cambiar el estado de necesidades	✅ Completa
F4 Correo	Bandeja de salida, worker, reintentos, webhook de Resend	✅ Completa
F5 Donantes	Registro, verificación de correo, perfil, altas asistidas	✅ Completa
F6 Invitaciones	Preselección, lotes, respuestas	🟡 Solo reglas de dominio (compatibilidad, distancia, ranking)
F7 Privacidad	Solicitudes de corrección y eliminación, accesos con motivo	✅ Completa
F8 Legal y reconocimientos	Información legal y políticas de reconocimiento	🟪 Pendiente
F9 Integración y salida	Conectar con el frontend, despliegue, mediciones	🟪 Pendiente

Avance del backend: ~69 % (6 completas y F6 al 20 %, sobre 9).

2.2 Frontend

Bloque	Estado
Pantallas web de Penpot: público, acceso, donante, hospital y superadmin	✅ Completo (con datos de prueba)
Revisiones de seguridad y clínica, y títulos por pantalla	✅ Completo
Estados de carga, error, sin permiso y conflicto	🟡 Donante listo; faltan hospital y superadmin
Inicio de sesión por rol	🟪 Pendiente (hoy hay un solo login con botones de prueba)
Conexión con el backend real	🟪 Pendiente
Versión para celular y tablet	🟪 Pendiente

Avance del frontend: ~55 %, ponderando las pantallas en 50 %, los estados en 15 %, el login por rol en 10 %, la integración en 15 % y el responsive en 10 %.

2.3 Avance global

~60 % del proyecto, promediando backend y frontend. Supera el 50 % previsto para esta entrega porque el frontend se adelantó con datos de prueba (ver la sección 6).

3. Lo implementado en esta etapa

- **Backend:**
  - 7 módulos en `backend/src/modules`;
  - 15 migraciones de base de datos con seguridad por fila (RLS);
  - worker de correo;
  - 66 operaciones de API ratificadas en `docs/openapi.json`.
- **Seguridad:** se corrigieron los hallazgos B1–B4, SP-01 (barrera de sesión tras cambiar la contraseña, ADR-007), SP-04 y P-12 (borrado de cuentas), con revisiones independientes aprobadas.
- **Frontend:**
  - todas las pantallas web de los 3 roles, siguiendo el diseño de Penpot;
  - capa de datos reemplazable (`demoApi`);
  - estados de error y conflicto en el área del donante;
  - accesibilidad verificada en el navegador.
- **Documentación:**
  - índice por libros en `docs/README.md`, con estado y fuente de verdad de cada documento;
  - sitio de documentación con VitePress (sección 9);
  - diagramas UML de arquitectura, patrones, clases y composición (25 páginas).
- **Calidad:**
  - **1091** pruebas automáticas que pasan (backend 799, frontend 227, mock 50 y web 15);
  - **0** vulnerabilidades en producción;
  - lint y verificación de tipos sin errores.

4. Arquitectura (visto en clase)

Estilo: monolito modular con arquitectura hexagonal (puertos y adaptadores).

Pieza	Tecnología	Dónde se publica
Frontend	React 18 + TypeScript + Tailwind	Vercel
API	Node.js + Express 5 (monolito modular)	Render
Worker de correo	Proceso Node aparte	Render
Base de datos y cuentas	Supabase (PostgreSQL, RLS, Auth con MFA)	Supabase

Correo	Resend, detrás de un adaptador	Resend
Documentación	VitePress sobre `docs/`	Local por ahora; publicación prevista en el Sprint 6

Por qué esta arquitectura (decisión documentada en `docs/decisions/ADR-001`):

- Un equipo pequeño y una demo de 3 hospitales no justifican microservicios.
- Un solo backend se publica de una vez y cada operación se guarda completa (una RPC = una transacción).
- Los módulos tienen límites claros y uno se puede separar si el proyecto crece.
- Se evaluó Java/Spring y se descartó: no había requisito que lo justificara y obligaba a usar dos lenguajes.

Evidencias:

- **1 backend** con **7 módulos**, cada uno dividido en `domain/`, `application/`, `http/` e `infrastructure/`.
- **0 dependencias** de las reglas (`domain/`) hacia Express, Supabase o Resend. Se verificó en el código.
- **7 ADR** con cada decisión importante, sus alternativas y el porqué.
- **Contract-first:** `docs/openapi.json` genera los tipos del backend, el mock y la web.

5. Patrones de diseño (visto en clase)

Patrón	Estado	Dónde (archivo)	Cómo se aplicó
Factory	Implementado	`backend/src/shared/persistence/supabase-client.ts`, `rpc-invoker.ts`	`SupabaseFactory` crea el cliente correcto: `forUser(token)` o `anonymous()`. Quien lo usa no sabe qué cliente concreto recibe
Adapter	Implementado	`email-delivery/infrastructure/resend-sender.ts`, `need-records/infrastructure/needs-rpc-gateway.ts`	Traducen las interfaces del dominio (`EmailSender`, `NeedsRepository`) a la API de Resend y a las RPC de Supabase
Strategy	Implementado	`email-delivery/application/email-sender.ts`, `infrastructure/fake-sender.ts`, `composition-worker.ts`	Dos estrategias de envío intercambiables, `FakeEmailSender` (pruebas) y Resend (producción), elegidas por configuración
Observer	Parcial	`backend/src/app.ts`, `server.ts`, webhook de Resend	Se usa el mecanismo de eventos de Node (`res.on('finish')`, `process.on('SIGTERM')`). Resend notifica por webhook
Decorator	Parcial	`platform/http/health.router.ts`, `app.ts`	`withTimeout` envuelve una promesa y le agrega un límite de tiempo; `requestLogger` envuelve cada petición

«Parcial» significa que se usan los mecanismos que ya trae Node, en vez de clases propias. Cumplen la misma idea sin código extra.

Otros patrones usados:

- Repository (`NeedsRepository`);
- inyección de dependencias y composition root (`composition.ts`);
- máquina de estados (`need-state.ts`, `invitation-state.ts`);
- Template (plantillas de correo);
- outbox transaccional;
- idempotency key;
- en el frontend: Provider (Context), custom hooks y render props (`QueryView`).

Diagramas de clases y UML: están en el repositorio, en `evidence_pc2/` (archivo editable `save-a-life-uml.drawio` y una imagen por diagrama), con su explicación en `evidence_pc2/diagramas_uml.md`: https://github.com/DiegoRomanP/Safe-a-Life/tree/main/evidence_pc2

Diagramas de clases	Qué muestran
20 · Modelo de dominio	Tipos y reglas puras (compatibilidad, distancia, estados)
21 · need-records	Necesidades: router, servicio, puerto `NeedsRepository` y adaptador RPC
22 · donor-records	Donantes: registro, perfil y altas asistidas
23 · identity-institutions	Sesión, MFA, tokens de acción e instituciones
24 · email-delivery	Correo: puerto `EmailSender`, Strategy (`FakeEmailSender` / Resend), outbox y worker
25 · privacy-requests	Solicitudes de privacidad y accesos contextuales
26 · shared	Piezas comunes: autenticación, HTTP, persistencia (`SupabaseFactory`) y criptografía
27 · Composición	Cómo `composition.ts` conecta cada servicio con sus adaptadores

Además hay 17 diagramas de arquitectura y patrones (despliegue, componentes, hexagonal, Strategy, outbox, máquina de estados, entre otros) en el mismo archivo.

6. Gestión del proyecto con Scrum (visto en clase)

6.1 Roles

Rol Scrum	Responsable
Product Owner	Team SaL (rol compartido por todo el equipo)
Scrum Master	Team SaL (rol compartido por todo el equipo)
Equipo de desarrollo	Diego Roman (backend, base de datos, seguridad), Jhan Leiva (frontend), Ator512 (configuración inicial y esquema de la base)

Por qué roles compartidos: es un proyecto de curso y el objetivo es que todos aprendan cada rol. Por eso la priorización del backlog (Product Owner) y la facilitación del proceso (Scrum Master) las asume el equipo completo, **Team SaL**, en las reuniones semanales.

6.2 Artefactos

Artefacto Scrum	Cómo se aplicó
Product Backlog	Requisitos de la ERS (`docs/2_Especificaciones_De_Software.md`), con identificadores RF/RNF, y tareas en Plane (por ejemplo `SAVELIFE-2`, `SAVELIFE-4`)
Sprint Backlog	Las tareas de cada ciclo en Plane, más un plan por tarea en `docs/plans/AAAA-MM-DD-<tarea>.md`

Incremento	Cada pull request mergeado en `main` con sus pruebas en verde
Definición de Hecho	`AGENTS.md`: pruebas, lint, typecheck y build en verde; revisión independiente aprobada; plan actualizado

6.3 Iteraciones (ciclos en Plane)

Iteración	Fechas	Objetivo	Resultado
Sprint 0 · Setup y prototipo	24–30 sep	Arranque: Express, Supabase, primer esquema, alcances	Repositorio, esquema inicial, PR #1 a #3
Sprint 1 · Especificación y backend base	1–14 oct	Especificación y backend base (F1–F5)	ERS, contrato OpenAPI, DTO, ADR, módulos F1 a F5
Sprint 2 · Seguridad y frontend (entrega del 50 %)	15–28 oct	Privacidad (F7), verificación de seguridad y frontend completo	F7, correcciones B1–B4, SP-01, SP-04 y P-12; PR #4 a #25 con todo el frontend. Cierra con la presentación del 50 %. Incluye el sitio de documentación con VitePress

Cada sprint es un **ciclo (cycle)** en Plane con sus tareas: Sprint 0 (7 tareas), Sprint 1 (11 tareas) y Sprint 2 (9 tareas). Las fechas coinciden con el historial de commits y pull requests de GitHub.

6.4 Evidencia en Plane (datos extraídos el 27 de octubre de 2026)

La vista pública de Plane muestra las tareas, pero no los ciclos ni las estadísticas. Por eso se incluyen aquí, tal como figuran en el proyecto.

Progreso por ciclo:

Ciclo	Fechas	Tareas hechas	Avance
Sprint 0 · Setup y prototipo	24–30 sep	7 de 7	100 %
Sprint 1 · Especificación y backend base	1–14 oct	11 de 11	100 %
Sprint 2 · Seguridad y frontend (entrega del 50 %)	15–28 oct	9 de 9	100 %

Estado de todas las tareas del proyecto (50 en total):

Estado	Tareas	Porcentaje
Hechas (Done)	27	54 %
En curso (In Progress)	2	4 %
Pendientes (Backlog)	21	42 %

El 54 % de tareas hechas coincide con el avance del 50 % que pide esta entrega. Las 21 tareas del backlog son las que se planificarán en los próximos sprints, hasta diciembre. Las cifras de Plane son las del 27 de octubre y no incluyen el trabajo de documentación con VitePress del 28 de octubre.

Tareas hechas por integrante (una tarea puede tener más de un responsable):

Integrante (Plane)	Tareas hechas

Diego Osvaldo Roman Palomino	13
Jhan	10
Vic	6

6.5 Ceremonias y prácticas

- **Planificación:** al inicio de cada iteración se eligen tareas del backlog. Cada una tiene su plan en `docs/plans`.
- **Revisión:** cada cambio entra por un pull request revisado. Los cambios sensibles pasan por revisores independientes de seguridad, de privacidad, clínico y de frontend/backend.
- **Integración continua:** GitHub Actions corre pruebas, contrato, build, gitleaks (detección de secretos) y npm audit en cada PR.
- **Retrospectiva:** los hallazgos de las revisiones generaron mejoras en el proceso, por ejemplo cómo crear datos de prueba sin que gitleaks los marque como secretos.
- **Seguimiento:** reuniones semanales del equipo, en lugar de la daily diaria, para revisar el avance, los bloqueos y las tareas de la semana.

6.6 Planificación frente a ejecución: cambios

Planificado	Lo que pasó	Por qué
El frontend esperaría a que el backend estuviera completo	El frontend avanzó en paralelo con datos de prueba y una capa reemplazable (`demoApi`)	No bloquear al equipo; cuando el backend esté listo, solo se cambia esa capa
Hacer F6 (invitaciones) después de F5	Se priorizó corregir hallazgos de seguridad (B1–B4, SP-01, SP-04, P-12) antes de F6	Los planes marcaban esos hallazgos como bloqueantes antes de F6
Usar `apps/web` como frontend	Se creó `frontend/` en TypeScript desde el diseño de Penpot; `apps/web` quedó como demo contra el mock	Mejor calidad, tipos del contrato y accesibilidad
Borrar cuentas de Auth en F7	Se implementó aparte, con una decisión explícita (P-12)	Requería una decisión de seguridad sobre el uso de `service_role`
Un solo login para todos los roles	Se mantiene por ahora; el login propio para hospital y superadmin queda para el próximo sprint	No estaba diseñado en Penpot
Documentación como carpeta de archivos Markdown	Se organizó por libros y se publica como sitio con VitePress	Un solo punto de entrada, con búsqueda y navegación, que sirva de documentación oficial
H2.4 Registrar la fecha de la última donación (backlog inicial)	Se excluye del alcance: el sistema no pide ni guarda esa fecha (RF-PRO-05)	No afirmar elegibilidad clínica ni guardar datos de salud; la aptitud la evalúa el hospital
H6.3 Mostrar donantes en un mapa (backlog inicial)	Se excluye: las coordenadas exactas no se muestran en ninguna pantalla	Minimizar datos sensibles de ubicación; la distancia se calcula solo en el backend

7. División del trabajo hasta diciembre

Sprint	Fechas aprox.	Objetivo	Responsable
Sprint 3	29 oct – 9 nov	F6 invitaciones y lotes (migración, RPC, rutas, pruebas); estados del frontend en hospital y superadmin	Backend: Diego · Frontend: Jhan
Sprint 4	10 – 23 nov	Login por rol; F8 legal y reconocimientos (en «borrador»); diagramas finales	Frontend: Jhan · Backend: Diego
Sprint 5	24 nov – 7 dic	F9: cliente HTTP, conexión front–back, protección de rutas por rol, pruebas de extremo a extremo	Equipo completo
Sprint 6	8 – 19 dic	Despliegue en Vercel, Render y Supabase; medición del matching (p95 menor a 2 s); versión responsive; publicación del sitio de documentación; documentación final y demo	Equipo completo

Esta planificación se cargará como ciclos en Plane; las fechas y los responsables pueden ajustarse ahí.

8. Riesgos y pendientes

- **Datos reales:** requieren revisión legal (Ley N.º 29733) y validación clínica de la tabla de compatibilidad. El prototipo no afirma cumplimiento legal ni clínico.
- **F6** es la parte más sensible: une matching, correo y privacidad, y necesita revisión clínica y de seguridad.
- **Integración:** conectar el frontend con el backend real exige el cliente HTTP con cookies y CSRF, y la protección de rutas por rol. El login ya usa la sesión y la consulta de CSRF en la rama de integración; falta enviar el token en las peticiones y conectar el resto de las vistas.
- **Dependencias:** tres PRs de Dependabot (ESLint 10, TypeScript 7, React 19) son incompatibles y deben cerrarse; `ajv-formats` 3 queda en revisión del backend.
- **Documentación:** el sitio VitePress no está desplegado ni tiene CI; el `postinstall` de `esbuild@0.21.5` (copia que trae VitePress) está bloqueado a la espera de decidir si se añade a `allowScripts` (ADR-002); los enlaces a archivos fuera de `docs/` no se validan en el build.

9. Documentación oficial con VitePress

La carpeta `docs/` estaba desordenada para servir como fuente de documentación. Se organizó y se configuró un sitio con **VitePress 1.6.4**, sin mover ni renombrar archivos, porque scripts, `AGENTS.md`, `CLAUDE.md` y `.opencode/` los citan por ruta.

9.1 Cómo se organizó

El índice `docs/README.md` es ahora una portada por **libros**. Cada documento figura con su tipo, su estado (Aprobado, Borrador, Vigente o Histórico) y la fuente de verdad que cubre.

N.º	Libro	Contenido
1	Producto y requisitos	Primeros alcances (línea base aprobada), ERS (borrador), requisitos normalizados, backlog
2	Decisiones de arquitectura	ADR-001 a ADR-007
3	Contrato de API	`openapi.json`, reglas de API, contratos backend, contrato de necesidad en borrador
4	Backend	Plan incremental, especificación faltante, Backend-Route, planes por módulo
5	Frontend	Entrega frontend, diseño de interfaz, planes de vistas e integración
6	Proceso y agentes	Flujo de trabajo de agentes, identidad de Engram, historial de planes
7	Diagramas	UML en `evidence_pc2/` (25 páginas) y su plan
8	Operación	Entorno local, mocks, comandos de verificación, límites
9	Pendientes	Bloqueos y decisiones abiertas

El índice incluye una guía de mantenimiento: un documento nuevo se añade a su libro en el mismo commit, declara su estado y no duplica reglas, sino que enlaza a la fuente de verdad. Los documentos que no declaraban estado (flujo de agentes, diseño de interfaz e identidad de Engram) recibieron una cabecera.

9.2 Qué ofrece el sitio

- Interfaz en español, con búsqueda local, barra lateral por libros y navegación superior.
- `docs/README.md` es la portada del sitio y sigue siendo el índice que lee GitHub.
- Las carpetas `decisions/`, `plans/` y `pending/` se listan solas: un documento nuevo en ellas aparece sin editar la configuración. Los de la raíz de `docs/` se añaden a mano.
- Los 56 archivos Markdown de `docs/` compilan sin errores.

9.3 Cómo levantarlo

Comando (desde la raíz)	Qué hace
`npm run docs:dev`	Servidor local con recarga en http://localhost:5170 (puerto fijo, distinto del 5173 de `npm run dev`)
`npm run docs:build`	Genera el sitio estático en `docs/.vitepress/dist/` (ignorado por git)
`npm run docs:preview`	Sirve el resultado del build en http://localhost:5171

Requiere las dependencias instaladas desde la raíz con `SCARF_ANALYTICS=false npm ci`. VitePress se añadió como devDependency fija en el único `package-lock.json`; no cambió ninguna versión ya existente.

9.4 También se actualizó

- El `README.md` de la raíz: backend y contrato actuales (58 paths, 73 operaciones, 66 ratificadas), persistencia por RPC y RLS, sección «Documentación» y los scripts `docs:*`.
- Dos correcciones para que compile: se escaparon `<…>` sin proteger en la especificación del backend y se corrigió un enlace roto en la guía de entrega del frontend.

10. Enlaces

- **GitHub:** https://github.com/DiegoRomanP/Safe-a-Life
- **Plane (vista pública, sin iniciar sesión):** http://159.195.245.124:75/spaces/issues/83d844c48b9b40e1b51df54042cc00f3
- **Eraser (diagramas):** https://app.eraser.io/workspace/iclH0MjPckR9i4f2cPaO
- **Documentación:** carpeta `docs/` del repositorio (ERS, ADR, contrato y planes), publicada localmente como sitio con `npm run docs:dev` (puerto 5170)